

Redes Neuronales Convolucionales (CNN)

Arquitectura, funcionamiento y aplicaciones

Autor: Jesús Emmanuel Martínez García

Agenda

- Introducción e intuición
- Componentes principales
- Funcionamiento matemático
- Tipos de capas y su rol
- Hiperparámetros clave
- Aplicaciones reales
- Limitaciones
- Buenas prácticas y recursos

¿Qué es una CNN?

- Es un tipo de red neuronal especializada en **procesar datos con estructura espacial** (como imágenes o video).
- Aprende **patrones locales** mediante operaciones de convolución.
- Reduce la cantidad de parámetros en comparación con redes densas.

Intuición básica

Una CNN aprende **características jerárquicas**:

- Capas iniciales: detectan **bordes y texturas**.
- Capas intermedias: detectan **formas y partes de objetos**.
- Capas profundas: detectan **conceptos completos** (como "cara" o "auto").

Estructura general de una CNN

1. **Capa de entrada:** imagen o matriz.
2. **Capas convolucionales:** extraen características.
3. **Pooling:** reduce tamaño y retiene información importante.
4. **Capas densas:** combinan características para clasificar.
5. **Capa de salida:** genera predicciones.

Concepto de convolución

Una **convolución** aplica un filtro (kernel) sobre la imagen para producir un **mapa de características**:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

- (I): imagen de entrada
- (K): kernel (filtro)
- (S): salida (feature map)

Filtros y mapas de características

- Los filtros se aprenden durante el entrenamiento.
- Cada filtro detecta un tipo distinto de patrón.
- La salida se denomina **feature map**.

Padding y Stride

- **Padding:** rellena los bordes para conservar tamaño.
- **Stride:** determina el paso del filtro sobre la imagen.

Parámetro	Efecto
<code>padding='same'</code>	Mantiene dimensiones
<code>padding='valid'</code>	Reduce dimensiones
<code>stride=1,2,...</code>	Aumenta salto entre posiciones

Función de activación

- Después de la convolución se aplica una **función no lineal** (generalmente ReLU).

$$f(x) = \max(0, x)$$

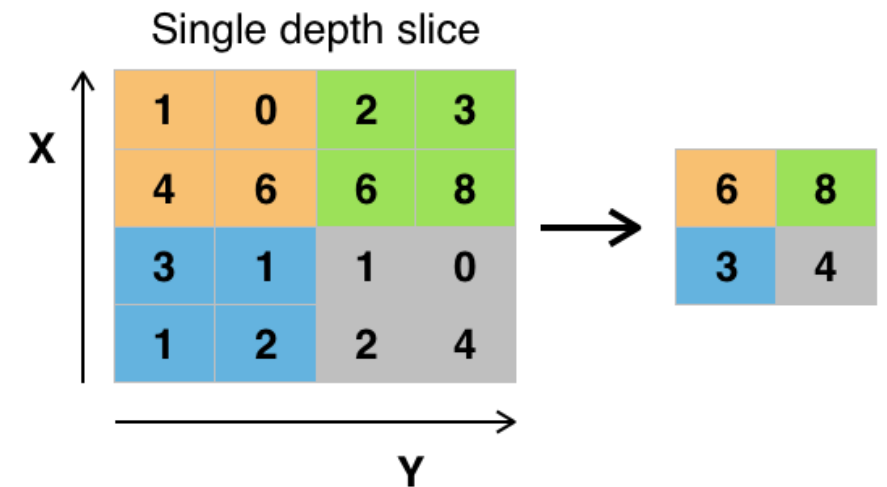
- Evita saturación y permite que la red aprenda representaciones complejas.

Capa de pooling

- Reduce la dimensionalidad conservando la información esencial.
- Ayuda a hacer la red **más robusta a traslaciones y ruido**.

Tipos comunes:

- **MaxPooling**: toma el valor máximo.
- **AveragePooling**: toma el promedio.



Capas densas (fully connected)

- Se colocan al final de la red.
- Combinan todas las características detectadas.
- Generalmente acompañadas de dropout para regularización.

$$y = \sigma(Wx + b)$$

Arquitectura clásica: LeNet-5

- Año: 1998 (Yann LeCun).
- Usada para reconocimiento de dígitos (MNIST).
- Estructura:
Conv → Pool → Conv → Pool → Dense →
Output.

Hiperparámetros principales

Hiperparámetro	Descripción	Ejemplo típico
Tamaño del filtro	Área del kernel	3x3, 5x5
Stride	Paso del filtro	1 o 2
Padding	Borde	'same'
Número de filtros	Profundidad	32, 64, 128
Batch size	Muestras por paso	32, 64
Tasa de aprendizaje	Paso del optimizador	1e-3 (Adam)
Dropout	Regularización	0.3–0.5

Arquitecturas modernas

Arquitectura	Innovación principal
AlexNet (2012)	Primera gran CNN en ImageNet
VGG (2014)	Capas pequeñas y profundas
GoogLeNet (2015)	Bloques Inception
ResNet (2016)	Conexiones residuales
EfficientNet (2019)	Escalamiento balanceado

Forward pass (propagación hacia adelante)

1. Imagen pasa por capas convolucionales → genera mapas de características.
2. Pooling → reduce dimensiones.
3. Flatten → convierte en vector.
4. Capas densas → clasifica.
5. Función de pérdida mide error.

$$\hat{y} = f(x; \theta)$$

Backpropagation en CNN

- Calcula gradientes de cada peso del kernel.
- Actualiza con optimizadores (SGD, Adam).
- Requiere **regla de la cadena** extendida para convoluciones.

Hiperparámetros adicionales

- **Número de capas:** controla profundidad.
- **Función de activación:** ReLU, LeakyReLU, GELU.
- **Optimizador:** Adam, RMSProp, SGD+Momentum.
- **Inicialización:** He Normal (para ReLU).
- **Regularización L2:** reduce sobreajuste.

Ejemplo de arquitectura CNN

Input(64x64x3)

→ Conv(32,3x3) + ReLU

→ MaxPool(2x2)

→ Conv(64,3x3) + ReLU

→ MaxPool(2x2)

→ Flatten

→ Dense(128) + ReLU + Dropout(0.5)

→ Dense(10) + Softmax

Función de pérdida: CrossEntropy

Optimizador: Adam

Batch size: 64

Épocas: 30

Aplicaciones principales

- **Visión por computadora:** clasificación, segmentación, detección.
- **Reconocimiento facial.**
- **Vehículos autónomos** (detección de objetos).
- **Medicina** (radiografías, tomografías).
- **Agricultura** (detección de plagas, cultivos).
- **Industria** (inspección de defectos).

Limitaciones

- **Altos requerimientos computacionales.**
- Necesitan **gran cantidad de datos etiquetados.**
- **Sensibles a rotaciones o escalas** si no se aumenta el dataset.
- **Difíciles de interpretar** (caja negra).
- Pueden **memorizar ruido** si no se regularizan bien.

Buenas prácticas

1. **Normalizar** imágenes (0–1 o -1–1).
2. **Data augmentation**: rotar, voltear, recortar.
3. Usar **Batch Normalization** para estabilidad.
4. **Dropout** para evitar sobreajuste.
5. Monitorear **accuracy y loss** por época.
6. Usar **transfer learning** con modelos preentrenados.
7. Aplicar **early stopping**.

Transfer Learning

- Reutiliza pesos de una CNN entrenada en un dataset grande (como ImageNet).
- Se reemplaza la última capa para adaptar a una nueva tarea.
- Ejemplo: ResNet, VGG, MobileNet.

Visualización e interpretabilidad

- **Mapas de activación:** muestran qué detecta cada filtro.
- **Grad-CAM:** resalta las regiones más relevantes para la predicción.

Métricas comunes

- **Accuracy:** porcentaje de aciertos.
- **Precision, Recall, F1:** para clases desbalanceadas.
- **IoU / Dice:** en segmentación.
- **Confusion Matrix:** para analizar errores.

Recursos recomendados

- **Libro:** *Deep Learning with Python* — François Chollet
- **Cursos:**
 - *Convolutional Neural Networks* (Andrew Ng - Coursera)
 - *Fast.ai Practical Deep Learning*
- **Frameworks:** TensorFlow, PyTorch, Keras
- **Herramientas:** TensorBoard, W&B, Grad-CAM

Conclusiones

- Las CNNs revolucionaron el procesamiento visual.
- Permiten **extraer características automáticamente**.
- Con buenos datos y regularización logran resultados sobresalientes.
- Su desafío actual: **eficiencia e interpretabilidad**.